UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3084 – Data Science

Sección 20

Ing. Mario Barrientos



Proyecto 2

Predicción de preferencias humanas en Chatbot Arena

Revisión bibliográfica y selección preliminar de algoritmos

Mia Alejandra Fuentes Mérida, 23775

Roberto Jose Barreda Siekavizza, 23354

Javier Eduardo España Pacheco, 23361

Angel Esteban Esquit Hernández, 23221

11 de octubre de 2026

Introducción

En la primera fase del Proyecto 2 se estudiaron las preferencias humanas entre respuestas de modelos de lenguaje grande (LLMs) usando el conjunto de datos de LMSYS Chatbot Arena. Cada registro presenta una pregunta o conversación, la respuesta del modelo A, la respuesta del modelo B y el voto de una persona: gana A, gana B o hay empate. La etapa anterior permitió comprender la información disponible; esta revisión se enfoca en elegir algoritmos para construir modelos que predigan esas preferencias.

La investigación reúne trabajos publicados en revistas científicas sobre clasificación de textos, modelos de aprendizaje supervisado y comparaciones por pares. A partir de esas fuentes se consideran alternativas que pueden implementarse en Python, desde métodos sencillos y fáciles de explicar hasta modelos que analizan el significado de las respuestas. La intención no es decidir anticipadamente cuál será el mejor, sino proponer una comparación justa y posible de ejecutar con los recursos del equipo.

Situación problemática

Evaluar la calidad de una respuesta automática es difícil porque no siempre existe una única respuesta correcta. Dos asistentes pueden responder adecuadamente una misma pregunta, pero una persona puede preferir uno por su claridad, utilidad, nivel de detalle o precisión. Por eso, el proyecto pretende aprender patrones de elección humana a partir del prompt, las dos respuestas y la información de los modelos enfrentados.

La dificultad principal es evitar conclusiones superficiales. La investigación anterior encontró una relación entre la extensión de las respuestas y los votos; sin embargo, una respuesta más extensa no necesariamente es mejor. Además, los empates son frecuentes y el historial de enfrentamientos de cada modelo no es igual. Un algoritmo útil debe aprender a distinguir estos factores y comprobar su capacidad de predicción con datos que no haya visto durante el entrenamiento (Equipo del Proyecto 2, 2026; Chiang et al., 2024).

Continuidad con el análisis exploratorio anterior

La selección de algoritmos se apoya en los resultados documentados por el grupo, no en resultados nuevos de entrenamiento. El estudio anterior reporta 57,477 enfrentamientos originales, con nueve columnas; después de retirar 71 duplicados de contenido se conservaron 57,406 registros y se construyeron variables numéricas para la siguiente etapa.

Hallazgo previo	Implicación para los modelos
Las clases se distribuyen en 34.94 % gana A, 34.22 % gana B y 30.85 % empate.	Se debe mantener una clasificación multiclase; no eliminar empates.
En batallas decisivas con respuestas de distinta longitud, gana la más larga en 61.63 % de casos.	Usar la diferencia de longitud como variable y contrastarla con información semántica.
La diferencia de longitud tiene correlaciones aproximadas de +0.177 con victoria A y −0.179 con victoria B.	Una característica aislada no basta: comparar modelos lineales y no lineales.
Hay 64 modelos distintos y desigual número de apariciones.	Incluir identidades de ambos modelos, y evitar interpretar el porcentaje bruto de victorias como calidad absoluta.
La ventaja global de posición es pequeña; no se demostró sesgo causal.	Conservar A/B como control y probar si el modelo cambia su decisión al invertir posiciones.

Las variables ya trabajadas incluyen model_a, model_b, prompt, response_a, response_b, prompt_len, resp_a_len, resp_b_len, prompt_words, resp_a_words, resp_b_words y n_turns. El resultado se puede

codificar como winner = 0 (A), 1 (B) o 2 (empate), tal como en el informe previo. La etiqueta se usa únicamente para aprender y evaluar, nunca como entrada del modelo.

Objetivos

Objetivo general

Revisar algoritmos de aprendizaje automático aplicables a la predicción de preferencias humanas entre dos respuestas de modelos de lenguaje, y seleccionar preliminarmente aquellos más adecuados para el conjunto de datos de Chatbot Arena.

Objetivos específicos

- Relacionar los hallazgos del análisis exploratorio anterior con posibles estrategias de clasificación y comparación por pares.
- Investigar al menos cinco algoritmos y sus ventajas, limitaciones y posibilidades de implementación en Python.
- Proponer modelos iniciales y una alternativa basada en contenido semántico, justificando la selección.
- Definir una metodología de evaluación que conserve la clase empate y evite fugas de información.

Marco teórico

Procesamiento del lenguaje natural (PLN)

El procesamiento del lenguaje natural es un área de la inteligencia artificial que permite analizar y representar textos. En este proyecto no basta con contar palabras: también interesa comprender si una respuesta aborda la pregunta, contiene información útil y se diferencia de la respuesta rival. Los modelos clásicos necesitan convertir los textos en números, mientras que los modelos modernos como BERT crean representaciones del contenido considerando su contexto (Minaee et al., 2021; Rogers et al., 2020).

Clasificación multiclase

La clasificación multiclase consiste en seleccionar una categoría entre tres o más opciones. Aquí las categorías son victoria de A, victoria de B y empate. En vez de reducir el problema a “gana o pierde”, se debe predecir también la tercera clase porque representa casi un tercio de los votos observados. Un modelo que solo prediga una de las dos victorias dejaría fuera una parte relevante del comportamiento humano.

Características manuales y representaciones del texto

Las características manuales son datos fáciles de calcular, como el número de caracteres, la cantidad de turnos o la diferencia de longitud entre dos respuestas. TF-IDF es una forma de representar la importancia de palabras dentro de muchos documentos. Los embeddings, en cambio, convierten textos en vectores que pueden conservar información de significado; por eso pueden ayudar cuando dos respuestas tienen longitudes parecidas pero contenido distinto (Minaee et al., 2021; Raffel et al., 2020).

Comparación por pares y empates

Los modelos de comparación por pares buscan describir cuál de dos alternativas suele preferirse. Bradley y Terry (1952) propusieron un modelo estadístico que expresa la probabilidad de que una alternativa venza a otra según su fuerza relativa. Davidson (1970) amplió este enfoque para incluir empates. Este tipo de método es relevante para Chatbot Arena porque cada observación compara exactamente dos modelos, aunque un método de ranking basado solo en identidades no logra analizar por sí mismo la calidad textual de cada respuesta.

Revisión bibliográfica de artículos de revistas indexadas

Se seleccionaron artículos de revistas científicas reconocidas en aprendizaje automático, estadística y PLN. Algunos estudian directamente comparaciones por pares o evaluación de LLMs; otros fundamentan algoritmos aplicables al problema. Esta distinción es importante: no se afirma que todos los artículos hayan utilizado el dataset LMSYS.

1. Bradley y Terry (1952) – comparaciones por pares

Biometrika. Proponen una forma de estimar qué alternativa tiene mayor probabilidad de ganar frente a otra. En Chatbot Arena serviría para estudiar la fuerza relativa de cada modelo, aunque su formulación básica no considera empates ni el contenido de las respuestas.

2. Davidson (1970) – incorporación de empates

Journal of the American Statistical Association. Presenta una extensión del modelo de comparación por pares que permite registrar "sin preferencia". Es especialmente pertinente porque el 30.85 % de los casos del conjunto limpio son empates.

3. Böhning (1992) – regresión logística multinomial

Annals of the Institute of Statistical Mathematics. Estudia un procedimiento de ajuste de modelos logísticos con varias categorías. Es una base apropiada para la línea de comparación que predice gana A, gana B o empate.

4. Breiman (2001) – Random Forest

Machine Learning. Estudia conjuntos de árboles de decisión que combinan predicciones y mejoran la estabilidad respecto de un árbol individual. La propuesta es útil para combinar longitudes, turnos e identidades de modelos sin suponer relaciones lineales.

5. Cortes y Vapnik (1995) – máquinas de soporte vectorial

Machine Learning. Desarrollan una estrategia de clasificación basada en separar ejemplos mediante una frontera de decisión. Su aplicación con vectores TF-IDF ofrece una alternativa tradicional de PLN para aprovechar las palabras de prompts y respuestas.

6. Friedman (2001) – gradient boosting

The Annals of Statistics. Presenta el aprendizaje progresivo de modelos que corrigen errores de modelos anteriores, incluyendo clasificación multiclase. Es la base teórica de familias actuales como XGBoost; el artículo no es una evaluación específica de XGBoost sobre Chatbot Arena.

7. Minaee et al. (2021) – clasificación profunda de textos

ACM Computing Surveys. Revisan múltiples arquitecturas de aprendizaje profundo para clasificación textual. Fundamentan que, además de los datos numéricos, se deben comparar enfoques capaces de extraer significado del contenido.

8. Rumelhart et al. (1986) – entrenamiento de redes neuronales

Nature. Explican el ajuste de pesos mediante retropropagación del error en redes de capas. Es una referencia clásica para considerar una red multicapa como alternativa de clasificación.

9. Rogers et al. (2020) – BERT y su funcionamiento

Transactions of the Association for Computational Linguistics. Explican qué tipo de información puede aprender BERT y sus limitaciones. Apoyan estudiar un clasificador que reciba conjuntamente pregunta y dos respuestas, en lugar de basarse únicamente en caracteres.

10. Raffel et al. (2020) – aprendizaje por transferencia

Journal of Machine Learning Research. Analizan cómo reutilizar modelos preentrenados en diversas tareas de lenguaje. Justifican adaptar un modelo de texto existente a la predicción de preferencias, evitando entrenar desde cero un modelo de gran tamaño.

11. Chang et al. (2024) – evaluación de LLMs

ACM Transactions on Intelligent Systems and Technology. Revisan dimensiones y métodos de evaluación de modelos de lenguaje. Su aporte es recordar que calidad, utilidad, seguridad y preferencia humana son conceptos relacionados pero no idénticos.

12. Liu et al. (2025) – conjuntos de datos de preferencias

Artificial Intelligence Review. Analizan distintas clases de datos de entrenamiento y evaluación de LLMs, incluidos conjuntos con comparaciones y votos humanos. Ayudan a ubicar la naturaleza del dataset de Chatbot Arena dentro de la investigación actual.

Algoritmos considerados para el proyecto

Se estudian seis familias principales y una variante de ranking. Todas son alternativas preliminares: su rendimiento en este proyecto se conocerá después de entrenarlas y compararlas.

1. Regresión logística multinomial

Funcionamiento. Aprende una relación entre las características de entrada y la probabilidad de cada uno de los tres resultados (Böhning, 1992). Es fácil de interpretar y permite construir una línea base que vaya más allá de predecir siempre la clase mayoritaria.

Aplicación al proyecto. Se considera para probar si longitudes, turnos y modelos enfrentados ya permiten anticipar preferencias. Si su rendimiento es bajo, habrá evidencia de que se necesita una relación más compleja o incorporar contenido textual.

Implementación en Python. scikit-learn: LogisticRegression; OneHotEncoder; StandardScaler.

Aspecto a considerar. Limitación: con variables simples solo captura relaciones principalmente lineales.

2. Random Forest

Funcionamiento. Combina numerosos árboles de decisión, cada uno entrenado con variaciones de los datos y las variables (Breiman, 2001). Puede descubrir interacciones entre longitud, número de turnos y modelos específicos.

Aplicación al proyecto. Es una comparación natural frente a la regresión logística: permite evaluar si las combinaciones de características del EDA mejoran la predicción de A, B o empate.

Implementación en Python. scikit-learn: RandomForestClassifier.

Aspecto a considerar. Limitación: no entiende por sí mismo el significado del texto y puede consumir más memoria.

3. Gradient Boosting (XGBoost)

Funcionamiento. Construye árboles de forma sucesiva para corregir errores anteriores. Friedman (2001) fundamenta este enfoque; XGBoost es una implementación eficiente posterior.

Aplicación al proyecto. Se considera para identificar relaciones no lineales entre las variables creadas y obtener un punto de comparación fuerte con datos tabulares. La salida será una clasificación de tres clases.

Implementación en Python. xgboost: XGBClassifier con objective="multi:softprob".

Aspecto a considerar. Limitación: necesita ajuste cuidadoso para reducir sobreajuste y requiere transformar categorías.

4. Máquinas de soporte vectorial (SVM)

Funcionamiento. Buscan separar categorías utilizando características numéricas del texto (Cortes & Vapnik, 1995). Con TF-IDF se representan palabras de los prompts y respuestas, sin usar redes neuronales grandes.

Aplicación al proyecto. Se considera para comprobar si el contenido expresado mediante palabras aporta más información que la longitud. Puede combinarse con los datos numéricos ya disponibles.

Implementación en Python. scikit-learn: TfidfVectorizer + LinearSVC; calibración si se necesitan probabilidades.

Aspecto a considerar. Limitación: TF-IDF no comprende profundamente el contexto; la variante lineal no entrega probabilidades directamente.

5. Red neuronal multicapa (MLP)

Funcionamiento. Recibe características numéricas o vectores de texto y aprende relaciones mediante varias capas conectadas (Rumelhart et al., 1986). Puede captar patrones más flexibles que un modelo lineal.

Aplicación al proyecto. Se incluye como opción intermedia entre modelos clásicos y Transformers; permitirá comprobar si más complejidad mejora la clasificación sin depender de un modelo lingüístico grande.

Implementación en Python. scikit-learn: MLPClassifier o PyTorch: torch.nn.

Aspecto a considerar. Limitación: exige normalización, ajuste de arquitectura y vigilancia del sobreajuste.

6. Transformer preentrenado (BERT o variante)

Funcionamiento. Modelos como BERT generan representaciones que consideran el contexto de las palabras. Pueden analizar el prompt y las respuestas A/B como un mismo problema de comparación textual (Rogers et al., 2020; Raffel et al., 2020).

Aplicación al proyecto. Es la opción más importante para una segunda fase, porque el EDA mostró que longitud y número de turnos no explican toda la preferencia. Se puede adaptar un modelo preentrenado con una salida de tres clases.

Implementación en Python. transformers + PyTorch: AutoTokenizer y AutoModelForSequenceClassification(num_labels=3).

Aspecto a considerar. Limitación: mayor costo de memoria y entrenamiento; entradas extensas requieren truncamiento o selección cuidadosa.

7. Bradley–Terry con extensión de Davidson

Funcionamiento. Modela la probabilidad de que un modelo venza a otro con base en su fuerza relativa. La extensión de Davidson permite también estimar probabilidad de empate (Bradley & Terry, 1952; Davidson, 1970).

Aplicación al proyecto. Se considera como referencia específica de comparaciones por pares y ayuda a controlar diferencias entre rivales. Su propósito es complementar la clasificación, no reemplazar el análisis del texto.

Implementación en Python. Python: NumPy/SciPy para estimación de parámetros; implementación propia validada.

Aspecto a considerar. Limitación: la formulación simple usa identidades y no reconoce la calidad específica de cada respuesta.

Comparación preliminar de modelos

Modelo	Entrada prevista	Ventaja principal	Costo relativo
Regresión logística	Variables numéricas y categorías codificadas	Línea base clara y rápida	Bajo
Random Forest	Numéricas y categorías codificadas	Interacciones no lineales	Medio
XGBoost	Numéricas y categorías codificadas	Buen candidato tabular	Medio
SVM + TF-IDF	Palabras de prompt y respuestas	Introduce el contenido textual	Medio
MLP	Variables escaladas / vectores	Compara modelos neuronales ligeros	Medio
BERT / variante	Prompt y respuestas tokenizadas	Considera significado y contexto	Alto / GPU útil
Bradley–Terry–Davidson	Identidades y enfrentamientos	Referencia de ranking y empates	Bajo–medio

El costo presentado es una estimación cualitativa, no una medición experimental. Depende de la cantidad de variables, la longitud de los textos, los parámetros y el hardware empleado.

Selección preliminar y justificación

Modelos principales propuestos

Se recomienda iniciar con regresión logística multinomial, Random Forest, XGBoost y SVM con TF-IDF. Los tres primeros se pueden comparar con el conjunto de características construido en el análisis exploratorio y la identidad de cada modelo; el cuarto incorpora palabras del texto sin el costo de entrenar un Transformer. El modelo de mayoría será una referencia mínima adicional, pero no se contará como algoritmo final.

Modelo de ampliación semántica

Si se dispone de GPU o tiempo suficiente, se propone un Transformer preentrenado para clasificación multiclase. Este modelo servirá para comprobar si leer conjuntamente pregunta y dos respuestas permite mejorar la predicción respecto a variables simples. No se plantea entrenar un LLM desde cero. La alternativa MLP puede utilizarse como control de complejidad intermedia.

Referencia para ranking y control de rivales

El modelo Bradley–Terry con empates de Davidson será una comparación complementaria, especialmente útil para analizar cuánto del resultado se explica por la identidad de los modelos enfrentados. Debe estimarse únicamente con los registros de entrenamiento de cada partición para no introducir información del conjunto de prueba.

La selección es provisional: no se puede afirmar que XGBoost, SVM o BERT obtendrán mejor F1, precisión o menor pérdida hasta realizar los experimentos. Su prioridad responde a características del problema, literatura revisada y costo esperado de implementación.

Metodología propuesta para implementar en Python

Preparación de las entradas

- Reutilizar el dataset limpio del repositorio y comprobar que el orden de las etiquetas winner (0=A, 1=B, 2=empate) se mantiene constante.
- Conservar prompt, response_a y response_b para modelos de texto, y calcular resp_a_len, resp_b_len, dif_len, número de turnos y longitudes transformadas con log(1 + x).
- Codificar model_a y model_b como variables categóricas para algoritmos tabulares. Escalar características solo cuando el método lo requiera, ajustando el preprocesamiento con los datos de entrenamiento.
- Probar dos grupos de variables: uno solo numérico/categórico y otro que agregue TF-IDF o representaciones semánticas; así se determina si el contenido verdaderamente aporta información adicional.
- No incluir winner_model_a, winner_model_b, winner_tie ni estadísticas de victoria calculadas usando todas las filas como predictores, porque revelan información del resultado.

Particiones y control de fuga de información

Como propuesta inicial, separar 70 % para entrenamiento, 15 % para validación y 15 % para prueba, conservando aproximadamente la proporción de las tres clases. Esta distribución todavía no corresponde a una ejecución. Si existen prompts duplicados o muy parecidos, agrupar sus registros al dividir los datos para evitar que conversaciones relacionadas queden simultáneamente en entrenamiento y prueba. La partición debe fijarse antes de entrenar cualquier transformación de vocabulario o modelo.

Como comprobación adicional, evaluar un escenario donde los pares de modelos o algunos prompts no aparezcan en el entrenamiento. Esto ayuda a saber si el clasificador realmente aprende patrones de respuestas o solo recuerda enfrentamientos vistos anteriormente.

Herramientas consideradas

Etapa	Bibliotecas de Python
Carga, limpieza y variables	pandas, numpy, json
Transformación y algoritmos clásicos	scikit-learn
Gradient Boosting	xgboost
Representaciones y Transformer	transformers, torch
Ajuste y evaluación	scikit-learn, scipy
Visualización y aplicación futura	matplotlib, seaborn, plotly; Streamlit como opción de interfaz

Estas son bibliotecas recomendadas, no una afirmación de que ya estén instaladas o implementadas en el repositorio. El repositorio revisado contiene los notebooks de carga, limpieza y análisis exploratorio, junto con módulos src/config.py y src/load.py; la etapa de modelado corresponde al trabajo posterior.

Evaluación prevista de los algoritmos

El problema es de clasificación multiclase. Por eso, la comparación no debe limitarse al porcentaje total de aciertos: también se necesita comprobar el resultado por clase y, especialmente, el comportamiento de los empates. Se recomienda usar el mismo conjunto de prueba, particiones y proceso de limpieza para todos los modelos.

Métrica	Qué permitirá comprobar
Exactitud (accuracy)	Proporción general de enfrentamientos bien clasificados.
F1 macro	Equilibrio del desempeño entre A, B y empate; no favorece una clase por ser más frecuente.
Precisión y recall por clase	Si el modelo acierta las victorias A/B y reconoce los empates.
Matriz de confusión	Errores concretos, como confundir empate con victoria A.
Log loss multiclase	Calidad de las probabilidades, cuando el algoritmo las ofrece o se calibran.
Tiempo y memoria	Costo de entrenamiento e inferencia para comparar eficiencia.

También se sugiere realizar una prueba de intercambio A/B: si se invierten las dos respuestas y las identidades, las probabilidades de ganar A y B deberían intercambiarse y la de empate mantenerse aproximadamente estable. La diferencia observada puede indicar dependencia indeseada de la posición. Los datos del informe anterior no demuestran por sí solos un sesgo causal.

Visualizaciones a desarrollar en la fase experimental

- Gráfica de barras con F1 macro, accuracy y tiempo de cada algoritmo.

- Matrices de confusión para identificar errores frecuentes en la clase empate.
- Comparación de desempeño usando solo longitud frente a longitud más texto.
- Gráfica de probabilidades para medir la confianza de cada predicción.

Conclusiones de la revisión bibliográfica

- El objetivo correcto del proyecto es predecir preferencias humanas entre dos respuestas de LLMs, no compradores recurrentes. Las tres clases deben conservarse porque los empates forman parte del problema.
- El análisis anterior justifica empezar con variables de longitud, turnos e identidades de modelos; no obstante, sus correlaciones moderadas indican que también conviene incorporar información del contenido textual.
- La literatura sustenta combinar líneas base simples, modelos de árboles, SVM para texto, Transformers preentrenados y un modelo de comparación por pares con empates. Cada alternativa responde a una necesidad distinta y debe compararse de manera justa.
- La selección preliminar prioriza regresión logística, Random Forest, XGBoost y SVM con TF-IDF; BERT, MLP y Bradley–Terry–Davidson quedan como ampliaciones o controles útiles según tiempo y recursos.
- Esta investigación constituye una propuesta teórica. La elección definitiva deberá basarse en resultados reproducibles, F1 macro, errores por clase, costo de cómputo y análisis del posible efecto de posición.

Link del repositorio

https://github.com/miafuentes30/CC3084-PRY2

Referencias

Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs: I. The method of paired comparisons. Biometrika, 39(3–4), 324–345. https://doi.org/10.1093/biomet/39.3-4.324

Böhning, D. (1992). Multinomial logistic regression algorithm. Annals of the Institute of Statistical Mathematics, 44(1), 197–200. https://doi.org/10.1007/BF00048682

Breiman, L. (2001). Random forests. Machine Learning, 45, 5–32. https://doi.org/10.1023/A:1010933404324

Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., Chen, H., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., & Xie, X. (2024). A survey on evaluation of large language models. ACM Transactions on Intelligent Systems and Technology, 15(3), Article 39. https://doi.org/10.1145/3641289

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20, 273–297. https://doi.org/10.1007/BF00994018

Davidson, R. R. (1970). On extending the Bradley-Terry model to accommodate ties in paired comparison experiments. Journal of the American Statistical Association, 65(329), 317–328. https://doi.org/10.1080/01621459.1970.10481082

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Liu, Y., Cao, J., Liu, C., Ding, K., & Jin, L. (2025). Datasets for large language models: A comprehensive survey. Artificial Intelligence Review, 58, Article 403. https://doi.org/10.1007/s10462-025-11403-7

Minaee, S., Kalchbrenner, N., Cambria, E., Nikzad, N., Chenaghlu, M., & Gao, J. (2021). Deep learning–based text classification: A comprehensive review. ACM Computing Surveys, 54(3), Article 62. https://doi.org/10.1145/3439726

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research, 21(140), 1–67. https://www.jmlr.org/papers/v21/20-074.html

Rogers, A., Kovaleva, O., & Rumshisky, A. (2020). A primer in BERTology: What we know about how BERT works. Transactions of the Association for Computational Linguistics, 8, 842–866. https://doi.org/10.1162/tacl_a_00349

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536. https://doi.org/10.1038/323533a0

Fuentes específicas del dataset y del trabajo anterior (no contabilizadas como artículos de revista)

Chiang, W.-L., Zheng, L., Sheng, Y., Angelopoulos, A. N., Li, T., Li, D., Zhu, B., Zhang, H., Jordan, M., Gonzalez, J. E., & Stoica, I. (2024). Chatbot Arena: An open platform for evaluating LLMs by human preference. En Proceedings of the 41st International Conference on Machine Learning (Vol. 235, pp. 8359–8388). Proceedings of Machine Learning Research. https://proceedings.mlr.press/v235/chiang24b.html

Equipo del Proyecto 2. (2026). Predicción de preferencias humanas en Chatbot Arena [Informe académico no publicado]. Universidad del Valle de Guatemala.

Kaggle. (2024). LMSYS - Chatbot Arena Human Preference Predictions [Competencia y conjunto de datos]. https://www.kaggle.com/competitions/lmsys-chatbot-arena